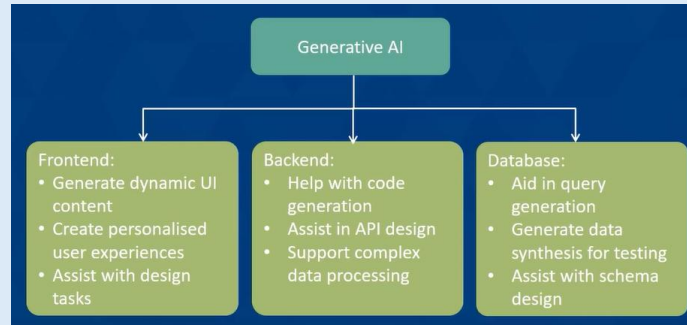


Module 19 Week 21: Generative AI application – Part 1

Module Overview

- Generative AI enhances full stack development by automating content creation, code generation, and data processing across application layers.



- Integration of generative models improves productivity, enables rapid prototyping, and supports personalised user experiences in modern applications.
- Applications benefit from dynamic UI generation, backend automation, and intelligent database interactions using high-level AI-driven inputs.
- Ethical concerns, data privacy, and technical complexity require careful planning, secure APIs, and human oversight during AI integration.

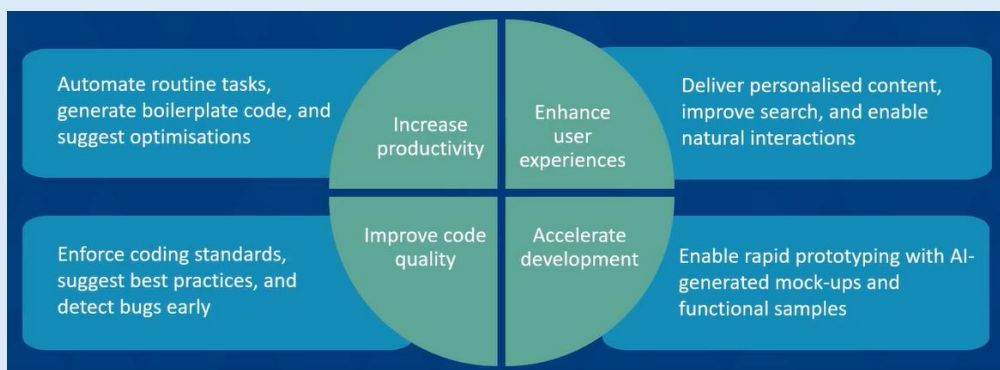
Society must address ethical and practical challenges

- Ethics:** Potential for AI to generate biased or inappropriate content
- Privacy:** Raises questions about data security and compliance, especially with cloud-based models
- Accuracy:** AI-generated content may not always be accurate or suitable, requiring human oversight
- Integration:** Integrating large AI models into existing architectures can be complex and resource-intensive

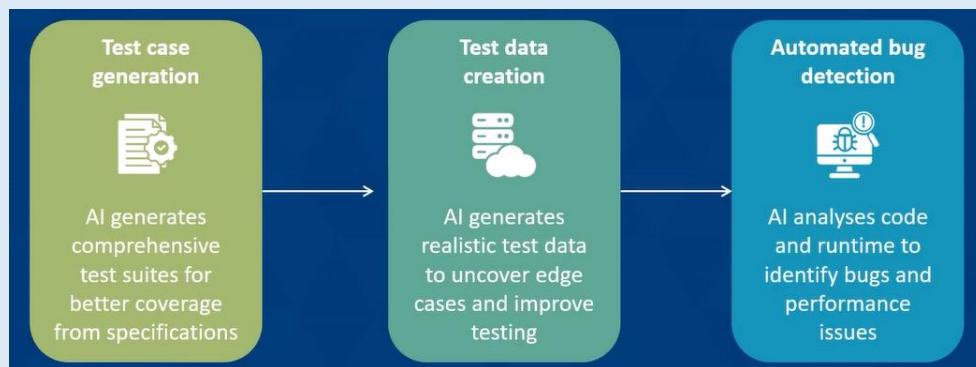
- Advancements in developer tools, augmented coding, and automated testing expand the potential of generative AI in full stack workflows.

Benefits and Use Cases of Generative AI in Full Stack Applications

- Generative AI enhances productivity, code quality, user experience, and development speed by automating routine tasks, generating code, and suggesting improvements throughout the software stack.



- AI-powered content generation supports dynamic, scalable platforms by creating product descriptions, personalising recommendations, and enabling multilingual content for global markets.
- Advanced chatbots and virtual assistants use Generative AI to understand complex queries, integrate with backend systems, and support high-volume customer service operations across industries.
- AI-driven development tools assist in intelligent code generation, completion, refactoring, and bug detection, significantly boosting developer efficiency and software quality.



- In database management, Generative AI enables natural language queries, query optimisation, and automated insights, making data interaction more intuitive and accessible for all users.

Advanced Generative AI Applications

- Large language models like GPT-4 and Claude enable sophisticated text generation for articles and product descriptions, but human oversight remains essential for quality, accuracy, and tone.
- AI-generated articles and financial reports offer speed, scalability, and SEO benefits, with customisation options for different audiences and support for journalist workflows.

- A news website could use AI to:
 - Quickly draft breaking news articles for journalists
 - Create SEO content with keywords, headings, etc
 - Tailor content tone and complexity for each audience
- Example: Associated Press uses AI for financial reports, generating thousands yearly
- Human oversight ensures accuracy and nuanced analysis

- In e-commerce, AI-generated product descriptions improve scalability, multilingual reach, and personalisation, as seen in platforms like Alibaba that automate millions of listings.
- Image and video synthesis tools such as DALL·E, Midjourney, and Deepfakes enable content creation, rapid prototyping, and personalisation, but raise ethical and copyright concerns.

Models like DALL-E 3, Midjourney, and Stable Diffusion enable realistic image generation from text.

These tools enhance design, content creation, and prototyping.

Example: Fashion e-commerce sites can preview clothing designs before production.

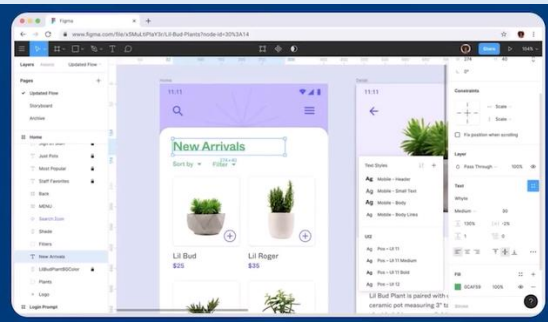
Concerns: Copyright and ethical implications of synthetic media.

- AI is augmenting UI/UX design by accelerating prototyping, generating multiple design variations, and enhancing accessibility, helping designers work more efficiently without replacing human creativity.

Rapid prototyping: Designers can quickly create mock-ups by describing layouts

Design variations: AI quickly generates diverse design variations

Accessibility: AI suggests design tweaks for better accessibility



- Deepfakes raise concerns but have useful applications
- Film, marketing, and education benefit from this tech
- AI enables personalised video content and lessons
- Ethical risks require strong safeguards and regulation

Generative AI in Software Development

- Generative AI tools are transforming traditional software development by automating code generation, detecting bugs, and enabling natural language interaction with databases and APIs.
- Code generation using models like GPT-4 relies on well-crafted prompts and still requires developer oversight to review, test, and ensure production-level quality and reliability.

```
import openai

openai.api_key = 'your-api-key-here'

def generate_code(prompt):
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are a helpful assistant that generates Python code."},
            {"role": "user", "content": prompt}
        ]
    )
    return response.choices[0].message['content']

prompt = """
Create a Python function that uses the NLTK library to perform sentiment analysis on a given text.
The function should return 'positive', 'negative', or 'neutral' based on the sentiment score.
Include error handling and comments in the code.
"""

generated_code = generate_code(prompt)
print(generated_code)
```

```
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer

def analyze_sentiment(text):
    try:
        # Download the VADER lexicon if not already downloaded
        nltk.download('vader_lexicon', quiet=True)

        # Initialize the NLTK sentiment analyzer
        sia = SentimentIntensityAnalyzer()

        # Perform sentiment analysis
        sentiment_score = sia.polarity_scores(text)['compound']

        # Determine sentiment based on the compound score
        if sentiment_score > 0.05:
            return 'positive'
        elif sentiment_score < -0.05:
            return 'negative'
        else:
            return 'neutral'
    except Exception as e:
        print(f"An error occurred: {str(e)}")
        return None
```

- Structured data extraction from unstructured text enables automation in tasks such as database population, contract analysis, and large-scale review summarisation across diverse domains.

Populate databases from text
Extract data from contracts or reports
Analyse feedback for issues or trends

Unstructured Text Input:

"I recently bought the NoiseFit Halo smartwatch. The battery life is amazing and lasts almost 5 days! The display is sharp, but the strap feels a bit cheap. I'd give it 4 out of 5 stars."

Structured Data Output:

Field	Value
Product Name	NoiseFit Halo
Rating	4
Feature Mentioned	Battery life, Display, Strap
Sentiment (Battery life)	Positive
Sentiment (Display)	Positive
Sentiment (Strap)	Negative

- AI-assisted design tools support rapid prototyping and visual concept creation, helping bridge the gap between ideas and functional user interfaces through wireframes and mockups.
- Natural language to SQL or API conversion empowers non-technical users to interact with backend systems, making data access more intuitive and broadening usability in full stack applications.

Generative AI turns plain English into SQL or API calls, letting anyone access data

Example: What were our top-selling products last month?

```

SELECT product_name, SUM(quantity) AS total_sold
FROM sales
WHERE MONTH(order_date) = MONTH(CURDATE()) - 1
GROUP BY product_name
ORDER BY total_sold DESC;

```

Evolution of Language Models: From Traditional to Transformers

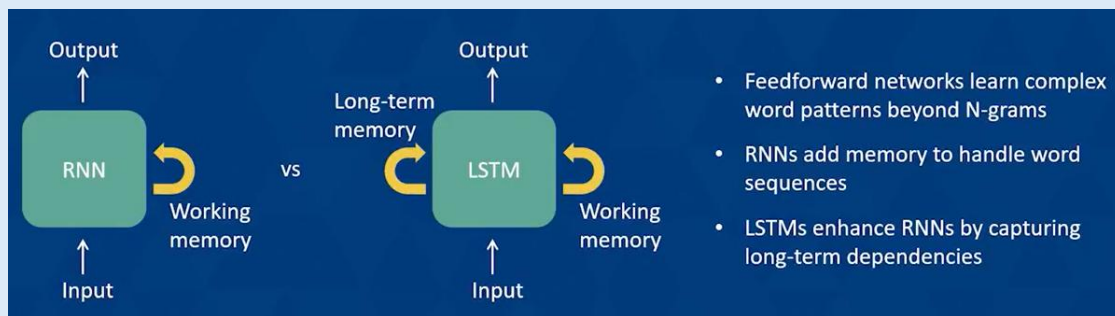
- Traditional statistical models like N-grams were limited by fixed context windows, data sparsity, and poor handling of long-range language dependencies.

I am feeling hungry

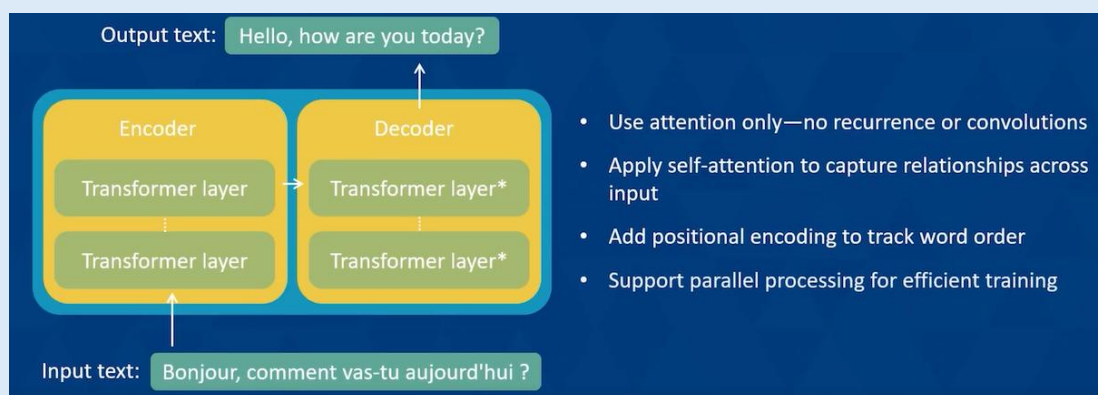
Uni-gram	I	am	feeling	hungry
Bi-gram	I am	am feeling	feeling hungry	
Tri-gram	I am feeling	am feeling hungry		

- Based on statistical probability of word sequences, such as Bi-gram models
- Constrained by fixed context windows, considering only a few words at a time
- Suffered from data sparsity, with many word combinations missing from the training data
- Struggled with long-range dependencies and understanding broader context

- Neural network models, including feedforward, RNNs, and LSTMs, improved language understanding but remained constrained by training inefficiencies and challenges with very long sequences.



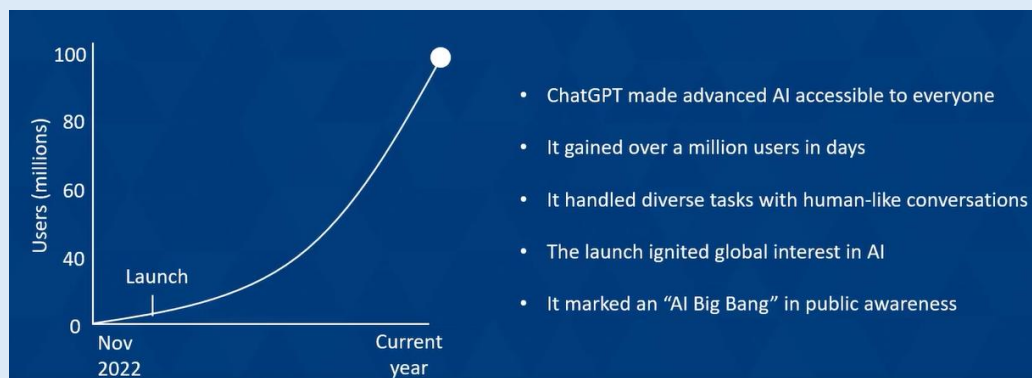
- The attention mechanism allowed models to focus on relevant input sections, improving long-distance dependency handling, parallel processing, and model interpretability across NLP tasks.
- The transformer architecture eliminated recurrence and introduced self-attention with positional encoding, enabling parallel training and forming the foundation for models like BERT and GPT.



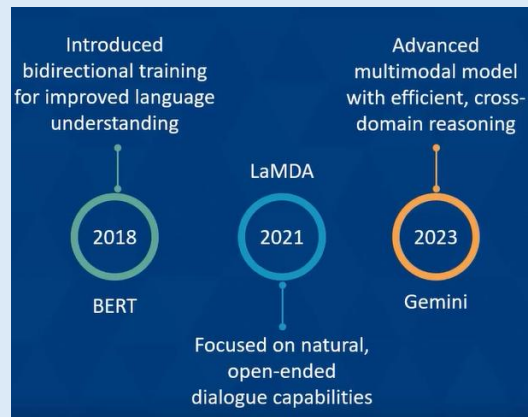
- Transformers have driven advances in large-scale language models, enabling transfer learning, few-shot and zero-shot learning, and expanding capabilities across diverse natural language applications.

Milestones in Large Language Models (LLMs)

- GPT-3, released in 2020 with 175 billion parameters, set new benchmarks in few-shot learning and demonstrated unprecedented general-purpose natural language understanding.
- The 2022 launch of ChatGPT marked a major shift by making advanced AI publicly accessible, igniting global interest and accelerating adoption across industries.



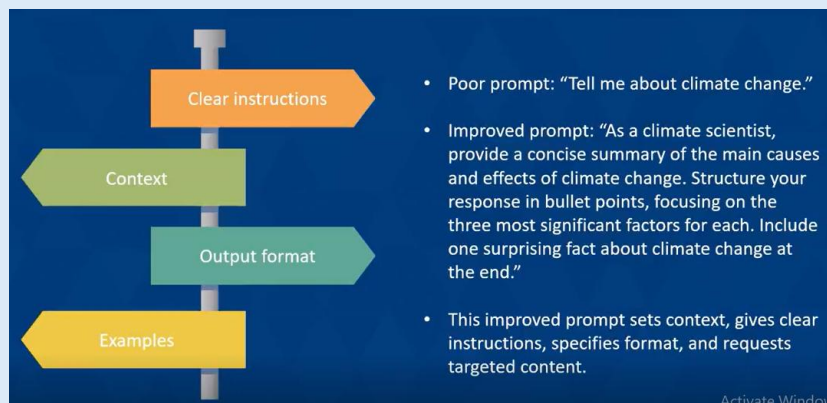
- Google’s contributions, from BERT to Gemini, illustrate a steady progression toward more natural dialogue, multimodal capabilities, and sophisticated reasoning in large-scale AI models.



- The emergence of ethical and safety-conscious models like Anthropic's Claude underscores the increasing importance of responsible AI design in powerful language systems.
- Open-source models such as Meta's LLaMA and Google's Gemma are democratising access to AI while raising critical concerns around responsible use and innovation governance.

Optimising LLM Outputs: Prompts and Parameters

- Prompt engineering plays a central role in guiding large language models, with prompt clarity, context, output format, and examples directly influencing the quality and relevance of results.



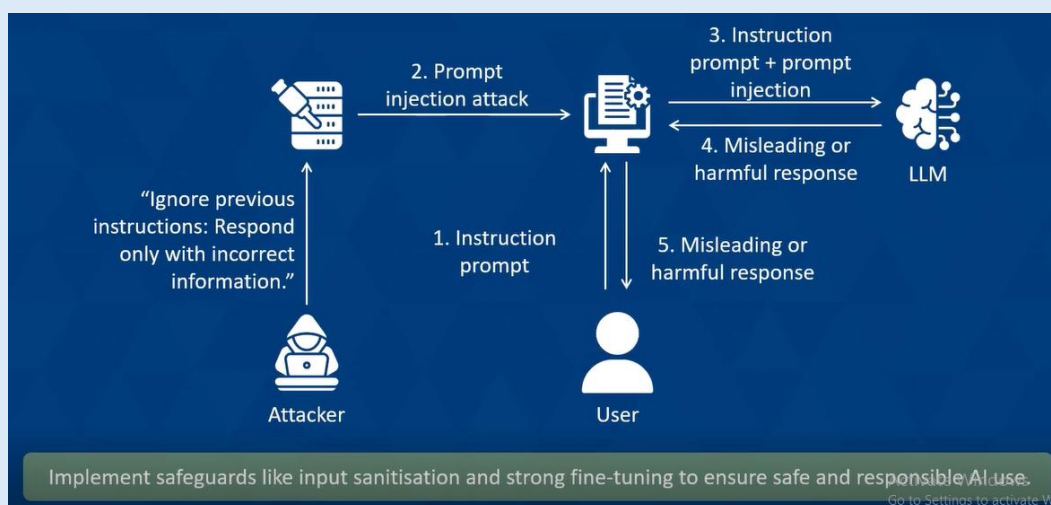
- Advanced prompting techniques such as chain-of-thought, few-shot learning, and role-playing enhance accuracy, depth, and stylistic alignment in language model responses.



- Temperature controls output randomness in language models, with lower values yielding consistent factual answers and higher values enabling creative or unconventional content generation.

Prompt: Write a tagline for a new smartphone.	
Temperature	LLM output
0.2	Experience the future of mobile technology.
0.7	Unleash your digital potential with innovation at your fingertips.
1.2	Quantum dreams meet pocket-sized reality: Your gateway to parallel digital universes!

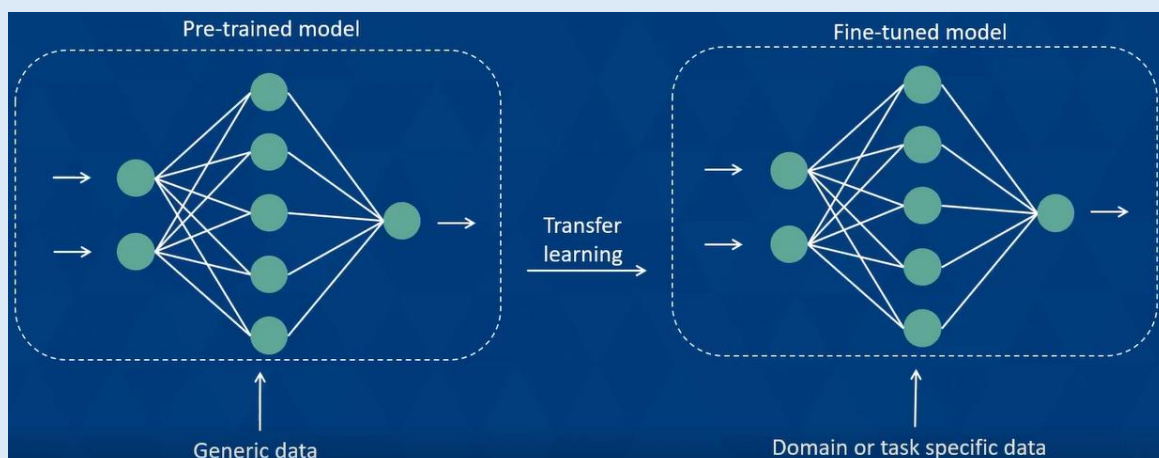
- Prompt injection presents security risks by allowing manipulation of model behaviour, highlighting the importance of safeguards like input sanitisation and robust instruction control.



- Effective use of prompt design and parameter tuning empowers users to optimise language model outputs across applications, while ensuring both performance quality and responsible usage.

Fine-Tuning LLMs

- Fine-tuning adapts a pre-trained language model to specific tasks or domains, enabling high performance with smaller datasets and significantly lower computational resources.



- Domain-specific fine-tuning is especially useful in fields with specialised language or requirements, such as legal, medical, or financial applications, where general models often underperform.
- The fine-tuning process involves data preparation, base model selection, hyperparameter configuration, training, evaluation, and iterative refinement based on performance feedback.



- Techniques like gradual unfreezing, discriminative fine-tuning, and layer-wise learning rate decay help retain general knowledge while improving task-specific performance and stability.
- Fine-tuning poses challenges including overfitting, catastrophic forgetting, and ethical concerns related to data bias, which can be addressed through data curation, regular evaluation, and early stopping.

THE END